

TemaFinale26 – Sprint 2 (v1)

Introduction

Questo documento (template del corso) descrive lo **Sprint 2** del sistema **cargoservice**. Lo Sprint 0 ([Sprint0_v1](#)) ha formalizzato requisiti e core business; lo Sprint 1 ([Sprint1_v1](#)) ha realizzato il sottosistema eseguibile core + sensor dell'IOPort + timeout 30s.

Goal dello Sprint 2 (sottoinsieme del testo del committente): *"It sends the answer retrylater, if ... the system is **Out of service**" e "the message '**Out of service**' if the sonar sensor measures a distance $D > D_{FREE}$ for at least 3 secs (perhaps a failure of the sonar)"* — piu' il **display** dell'IOPort realizzato come **web-gui** (domanda aperta D5 dello Sprint 1, ora chiusa).

Sintesi dello Sprint precedente (punto di partenza)

Il prototipo dello Sprint 1 e' **eseguito e testato**: ciclo nominale loadrequest → reserved(slot) → containerInPlace → IOPort → slot5(marcatura) → slot riservato → HOME, timeout 30s → disengage, gestione moverobotfailed. Architettura: cargoservice (QActor) client di robotsmart (ExternalQActor, contratto moverobot(X,Y,STEPTIME)), POJO Hold testato (20 PASS). Modello: [Sprint1/cargoservice26.qak](#).

Requirements

Testo esatto del committente in [Sprint 0 \(v1\) – Requirements](#). Requisiti oggetto di questo Sprint:

- *"It sends the answer **retrylater**, if the IOPort is currently occupied by a container or if the system is **Out of service**."*
- *"The service must also show on the display on the IOPort: ... the message '**Out of service**' if the sonar sensor measures a distance $D > D_{FREE}$ for at least 3 secs (perhaps a failure of the sonar)."*
- *"The display is used to show the answer to the request and to show the current state of the hold."*

Requirement analysis

Approfondimento del sottoproblema dello Sprint (le entita' generali sono nello Sprint 0):

Termine	Analisi
DFREE	il testo non lo fissa : e' un parametro di calibrazione del dispositivo (qui 60 cm, da tarare sul dispositivo reale col committente)

"for at least 3 secs"	condizione sostenuta nel tempo , non sul singolo campione: serve chi osserva il flusso di distanze e ne deriva lo stato
Out of service	stato del sistema (non del solo sensor): le richieste ricevono retrylater ; il display lo mostra. Il testo non definisce il rientro : si assume (decisione, da confermare col committente) il rientro quando $D \leq D_{FREE}$ in modo sostenuto (~3s)
display	mostra risposta e stato corrente: realizzato come web-gui (browser), coerente con la natura distribuita del sistema

Problem analysis

Architettura logica del sottosistema

Lo Sprint 1 aveva astratto il sensor nella notifica logica **containerInPlace**. Con l'Out of service serve anche il **dato grezzo**: la distanza D. Si introduce quindi la separazione (architettura logica):

```

iosensor (PicoW reale o simulato)
  | Event sonaralarm : distance(D)   (dato GREZZO, formato del PicoW reale)
  v
SENSORMONITOR -- D < DFREE/2 per ~3s --> containerInPlace --> CARGOSERVICE
               -- D > DFREE per >=3s --> outofservice(on) --> (stato outOfService:
               -- D ≤ DFREE per ~3s --> outofservice(off) --> retrylater + display)

```

Scelta	Motivazione
sonaralarm : distance(D) come Event	formato compatibile col PicoW reale (msg(sonaralarm,event,picow,none,distance(D),0), vedi Picow/Sonar/sonarMqtt.py): il dispositivo reale sostituisce il simulato senza modifiche al resto del sistema
sensormonitor attore dedicato	la logica "sostenuta per 3s" e' stato + tempo : natura da attore; separa il livello dispositivo dal livello servizio (colma l'abstraction gap)
notifiche logiche come Dispatch (point-to-point)	alternativa già prevista nell'analisi dello Sprint 1 ("notifica → evento, alt.: dispatch"): il destinatario e' uno solo (cargoservice) e si evita il broadcast del flusso di eventi a tutti gli attori
display web-gui via MQTT/websocket	mosquitto espone websocket (porta 9001, già nel deployment del corso): la pagina si sottoscrive al topic del display; il cargoservice non cambia (dipende solo dall'informazione: display.show(msg))

Comportamento atteso (livello di problema)

```

disponibile --(outofservice(on))--> OUT OF SERVICE: display "Out of service"
OUT OF SERVICE --(loadrequest)--> answer(retrylater)   (requisito)
OUT OF SERVICE --(outofservice(off))--> disponibile: display "serviceworking"
(il resto del comportamento e' quello dello Sprint 1)

```

Piano di lavoro (previsione, ore/uomo)

Attività	Previsione	Effettivo
analisi (sensormonitor, Out of service, rientro)	2 h	~2 h

modello qak + codice (sensormonitor, stati OOS, scenario demo)	3 h	~3 h
display web-gui (DisplayMqtt + pagina)	2 h	~1.5 h
test (unita' + end-to-end con robotsmart/VirtualRobot)	2 h	~2.5 h
totale	9 h	~9 h (tempi rispettati)

Distribuzione dei compiti: progetto **individuale** (Alexander Kreuzer): analisi, sviluppo e test svolti dalla stessa persona.

Diario di bordo dello Sprint

Data	Attivita'
09/06	consegnati Sprint0_v1/Sprint1_v1 (struttura in Sprint approvata dal docente); prototipo Sprint 1 eseguito end-to-end con robotsmart/VirtualRobot
10/06	analisi Out of service (DFREE, finestra 3s, rientro); introdotto il sensormonitor e il dato grezzo <code>distance(D)</code> nel formato del PicoW reale
10/06	display come web-gui (DisplayMqtt + pagina mqtt/websocket :9001); scenario demo TP1+TP5+TP6; build e demo eseguite; documento Sprint2_v1

Test plans

TP1 CARICO NOMINALE (Sprint1, regressione): loadrequest -> reserved(slot1); containerInPlace; IOPort -> slot5(marcatura) -> slot1 -> HOME; PASS sse "richiesta completata: container in slot1"

TP4 TIMEOUT (Sprint1, regressione): nessun container in 30s -> disengage; slot liberato

TP5 OUT OF SERVICE (questo Sprint):
pre : sistema disponibile
azione: il sensor misura $D=90 > DFREE$ per ~3s
attesi: display "Out of service" ; loadrequest -> answer(retrylater)
PASS sse: la risposta alla richiesta durante il guasto e' retrylater e il display mostra Out of service

TP6 RIENTRO IN SERVIZIO (questo Sprint):
azione: D rientra ($\leq DFREE$) per ~3s
attesi: display "serviceworking" ; loadrequest -> answer(reserved(slot2)) ; ciclo 2 completo
PASS sse: "richiesta completata: container in slot2"

La **demo significativa** esegue TP1+TP5+TP6 in un'unica run (scenario dell'iosensor):
ciclo nominale → guasto → retrylater → rientro → secondo ciclo.

Project

Sottosistema eseguibile: [TemaFinale26/Sprint2/cargoservice26](#) — modello [cargoservice26.qak](#):

Componente	Realizzazione
cargoservice	FSM Sprint1 + stati <code>checkOos</code> / <code>outOfService</code> / <code>oosRetry</code> / <code>checkOosExit</code> ; le notifiche logiche arrivano come dispatch (<code>whenMsg</code>)
sensormonitor (nuovo)	Sensormonitor.kt : contatori di campioni consecutivi (finestra ~3s a ~1Hz); forward al cargoservice

iosensor (simulato)	emette <code>sonaralarm:distance(D)</code> secondo lo scenario della demo (presenza/guasto/rientro/presenza); sostituibile dal PicoW reale
display web-gui	DisplayMqtt.java pubblica su MQTT (<code>cargoservice26display</code>); webgui/display.html (<code>mqtt.js</code> su websocket :9001) mostra stato e log nel browser
pushbutton (simulato)	3 richieste: nominale / durante il guasto / dopo il rientro

Robustezza del trasporto: un singolo passo del DDR puo' fallire (`moverobotfailed`, es. frame della scena WebGL in ritardo): il `cargoservice` **riprova lo stesso target fino a 2 volte** (stati `retryIOPort/retrySlot5/retryReserved`; `robotsmart` ripianifica dalla posizione corrente) e solo dopo libera lo slot (`robotFailure`), restando disponibile.

Limite noto (scelta di Sprint): `outofservice` e' gestito negli stati `available/outOfService`; durante un trasporto il dispatch resta in coda ed e' servito al ritorno in `available`. Copertura con `whenInterruptEvent`: Sprint 3.

Testing

POJO: HoId invariato, test di regressione [TestHold](#) = **20 PASS, 0 FAIL**. Build: `gradlew build` = BUILD SUCCESSFUL.

Demo eseguita (TP1+TP5+TP6 in un'unica run, con `robotsmart` + `VirtualRobot` + `mosquitto`):

```
pushbutton | premuto (1) -> loadrequest      -> risposta (1): reserved(slot1) [engaged, LED on]
sensormonitor | presenza container -> containerInPlace
cargoservice | trasporto IOPort -> slot5 (marcatrice) -> slot1 -> HOME
cargoservice | richiesta completata: container in slot1 [TP1 PASS]
sensormonitor | D>DFREE per ~3s -> outofservice(on)
cargoservice | OUT OF SERVICE ; [DISPLAY] Out of service
pushbutton | premuto (2) -> loadrequest      -> risposta (2): retrylater [TP5 PASS]
sensormonitor | sensor rientrato -> outofservice(off) ; [DISPLAY] serviceworking
pushbutton | premuto (3) -> loadrequest      -> risposta (3): reserved(slot2)
cargoservice | richiesta completata: container in slot2 [TP6 PASS]
```

Deployment

```
(da TemaFinale26/Sprint2/cargoservice26/)
1) docker compose -f ../../robotsmart26/yamls/unibobasic26.yaml up (VirtualRobot + GUI + mosquitto)
2) aprire http://localhost:8090 nel browser (SCENA WebGL del VirtualRobot - OBBLIGATORIO:
    la scena ESEGUE i movimenti; senza pagina aperta
    i moverobot non terminano mai)
3) cd ../../robotsmart26 ; ./gradlew run (servizio del docente, 8020)
4) ./gradlew run (cargoservice26, ctxcargo 8030)
5) aprire webgui/display.html nel browser (display IOPort web-gui, ws://localhost:9001)
```

NB (verificato durante il testing): la scena WebGL (punto 2) va aperta **prima** di lanciare le richieste e va **riaperta/ricaricata** se si riavvia il container wenv (la pagina si registra alla connessione). Con mosquitto in Docker usare l'**IP reale** del PC (non localhost) quando i dispositivi sono su altri host (es. PicoW reale).

Maintenance

Il cargoservice dipende solo dall'**informazione** scambiata: l'**iosensor** simulato si sostituisce col **PicoW reale** (stesso messaggio `distance(D)`) e il display a video con la **web-gui** senza toccare la FSM. DFREE e la finestra dei 3s sono **parametri** del sensormonitor.

Sintesi finale dello Sprint e goal del prossimo

Sprint	Stato
Sprint 2 (questo)	completato: Out of service + rientro + display web-gui; demo TP1+TP5+TP6; tempi rispettati (~9 h come previsto)
Goal Sprint 3	rifiniture: marker come attore (evento <code>markingDone</code> al posto del <code>delay</code>), gestione richieste durante il trasporto (<code>retrylater</code> immediato), <code>whenInterruptEvent</code> per outofservice in ogni stato, barcode; slides (<10) e preparazione della demo finale



By Alexander Kreuzer

Email: Alexander.kreuzer@studio.unibo.it

GIT repo: https://github.com/Alexing-unt/ISS26_Alexander_Kreuzer